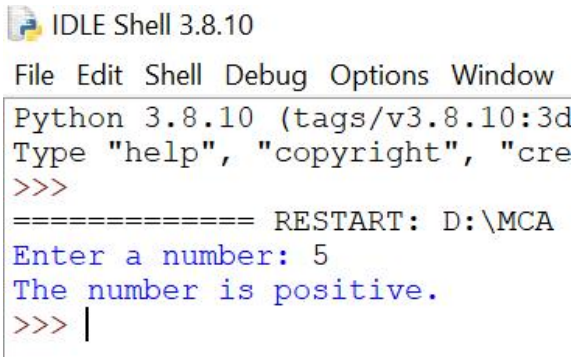
SOURCE CODE

1. Conditional Statements

```
num = float(input("Enter a number: "))
if num > 0:
    print("The number is positive.")
elif num < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

OUTPUT



RESULT

The given program was executed successfully.

SOURCE CODE

2. Triangle Pattern

```
rows = int(input("Enter number of rows: "))
for i in range(1,rows+1):
    for j in range(1,i+1):
        if j==4:
            break
        if j==2:
            continue
        if j==3:
            pass
        print(j,end="")
    print()
```

OUTPUT

RESULT

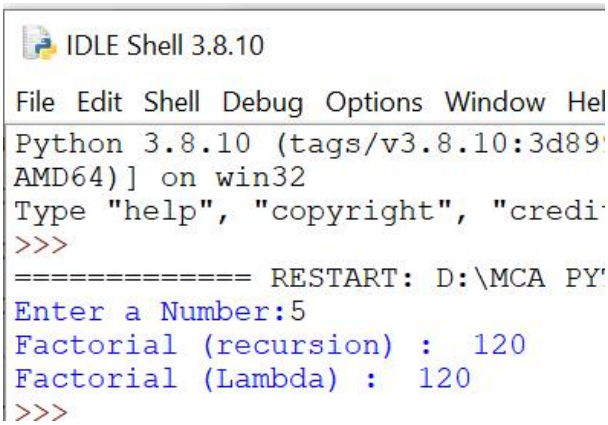
The given program was executed successfully.

SOURCE CODE

3. Recursive Functions and Lambda with Reduce()

```
from functools import reduce
def fact(n):
    if n==0 or n==1:
        return 1
    else:
        return n* fact(n-1)
num=int(input("Enter a Number:"))
print("Factorial (recursion) : ",fact(num))
factorial = reduce(lambda x,y:x*y,range(1,num+1))
print("Factorial (Lambda) : ",factorial)
```

OUTPUT



RESULT

The given program was executed successfully.

SOURCE CODE

4. Mini Banking System

Main.py

```
import bnk
while True:
    print("\n--- Mini Banking System ---")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Balance Inquiry")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    try:
        if choice == 1:
            amount = float(input("Enter deposit amount: "))
            print("Balance after deposit:", bnk.deposit(amount))
        elif choice == 2:
            amount = float(input("Enter withdrawal amount: "))
            print("Balance after withdrawal:", bnk.withdraw(amount))
        elif choice == 3:
            print("Current Balance:", bnk.check_balance())
        elif choice == 4:
            print("Thank you for using the banking system.")
            break
        else:
            print("Invalid choice!")
    except Exception as e:
        print("Error:", e)
```

bnk.py

```
balance = 0
def deposit(amount):
        global balance
        balance += amount
        return balance
def withdraw(amount):
```

```
        global balance
        if amount > balance:
            raise Exception("Insufficient balance!")
            balance -= amount
            return balance
def check_balance():
        return balance
```

OUTPUT

```
*IDLE Shell 3.8.10*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.8.10 (tags/v3.8.10:3d8993a, May  3 2021, 11:48:03) [MSC
Type "help", "copyright", "credits" or "license()" for more info
>>>
======== RESTART: D:\MCA PYTHON\PROGRAMS\Lab Programs\Pro 4\Pro

--- Mini Banking System ---
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 1
Enter deposit amount: 10000
Balance after deposit: 10000.0

--- Mini Banking System ---
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 2
Enter withdrawal amount: 2000
Balance after withdrawal: 8000.0

--- Mini Banking System ---
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: 3
Current Balance: 8000.0

--- Mini Banking System ---
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter your choice: |
```

RESULT

The given program was executed successfully.

SOURCE CODE

5. Employees Performance Using Conditional Logic

```
class employee:
    def __init__(self,emp_id,name,salary,performance):
        self.emp_id=emp_id
        self.name=name
        self.salary=salary
        self.performance=performance
    def bonus_calculate(self):
        if self.performance=="Excellent":
            bonus=self.salary*0.20
        elif self.performance=="Good":
            bonus=self.salary*0.10
        elif self.performance=="Average":
            bonus=self.salary*0.05
        else:
            bonus=0
        return bonus
    def display(self):
            print("Employee Id:",self.emp_id)
            print("Employee Name:",self.name)
            print("Employee Salary:",self.salary)
            print("Employee Performance:",self.performance)
            print("Employee Bonus:",self.bonus_calculate())
            emp_id=int(input("Enter Employee ID:"))
            name=input("Enter Employee Name:")
            salary=int(input("Enter Employee Salary:"))
            performance=input("Enter Employee Performance(Excellent/Good/Average):")
            emp=employee(emp_id,name,salary,performance)
emp.display()
```

OUTPUT

```
IDLE Shell 3.8.10
File  Edit  Shell  Debug  Options  Window  Help
Python 3.8.10 (tags/v3.8.10:3d8993a, May  3 2021, 11:48:03) [MSC v.1928 6
Type "help", "copyright", "credits" or "license()" for more information.
>>>
============= RESTART: D:\MCA PYTHON\PROGRAMS\Lab Programs\Pro 5.py =====
Enter Employee ID:1
Enter Employee Name:Alagiri
Enter Employee Salary:45000
Enter Employee Performance(Excellent/Good/Average):Good
Employee Id: 1
Employee Name: Alagiri
Employee Salary: 45000
Employee Performance: Good
Employee Bonus: 4500.0
```

RESULT

The given program was executed successfully.

SOURCE CODE

6. Book and Library classes

```
class Book:
    def __init__(self, book_id, title, author):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.is_available = True
    def display_book(self):
        status = "Available" if self.is_available else "Borrowed"
        print(f"ID: {self.book_id}, Title: {self.title}, Author: {self.author}, Status: {status}")
class Library:
    def __init__(self):
        self.books = []
   def add_book(self, book):
        self.books.append(book)
        print("Book added successfully!")
    def search_book(self, title):
        found = False
        for book in self.books:
            if book.title.lower() == title.lower():
                book.display_book()
                found = True
        if not found:
            print("Book not found.")
    def borrow_book(self, book_id):
        for book in self.books:
            if book.book_id == book_id:
                if book.is_available:
                    book.is_available = False
                    print("Book borrowed successfully!")
                else:
                    print("Book is already borrowed.")
                return
```

```
        print("Book ID not found.")
    def return_book(self, book_id):
        for book in self.books:
            if book.book_id == book_id:
                if not book.is_available:
                    book.is_available = True
                    print("Book returned successfully!")
                else:
                    print("Book was not borrowed.")
                return
        print("Book ID not found.")
    def display_all_books(self):
        if not self.books:
            print("No books in library.")
        for book in self.books:
            book.display_book()
library = Library()
while True:
    print("\n--- Library Menu ---")
    print("1. Add Book")
    print("2. Search Book")
    print("3. Borrow Book")
    print("4. Return Book")
    print("5. Display All Books")
    print("6. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        book_id = int(input("Enter Book ID: "))
        title = input("Enter Book Title: ")
        author = input("Enter Author Name: ")
        library.add_book(Book(book_id, title, author))
    elif choice == "2":
        title = input("Enter book title to search: ")
        library.search_book(title)
```

```
    elif choice == "3":
        book_id = int(input("Enter Book ID to borrow: "))
        library.borrow_book(book_id)
    elif choice == "4":
        book_id = int(input("Enter Book ID to return: "))
        library.return_book(book_id)
    elif choice == "5":
        library.display_all_books()
    elif choice == "6":
        print("Exiting Library System. Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT

```
*IDLE Shell 3.8.10*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.8.10 (tags/v3.8.10:3d8993a, May  3 2021, 11:48:03) [MSC v.1928 64 bit (AMI
Type "help", "copyright", "credits" or "license()" for more information.
>>>
============= RESTART: D:\MCA PYTHON\PROGRAMS\Lab Programs\Pro 6.py ============

--- Library Menu ---
1. Add Book
2. Search Book
3. Borrow Book
4. Return Book
5. Display All Books
6. Exit
Enter your choice: 1
Enter Book ID: 24
Enter Book Title: OOPS with C++
Enter Author Name: Balagurusamy
Book added successfully!

--- Library Menu ---
1. Add Book
2. Search Book
3. Borrow Book
4. Return Book
5. Display All Books
6. Exit
Enter your choice: 5
ID: 24, Title: OOPS with C++, Author: Balagurusamy, Status: Available

--- Library Menu ---
1. Add Book
2. Search Book
3. Borrow Book
4. Return Book
5. Display All Books
6. Exit
Enter your choice: |
```

RESULT

The given program was executed successfully.

SOURCE CODE

7. Student Details Using File

```
file_name = "students.txt"
def add_student():
    roll = input("Enter Roll Number: ")
    name = input("Enter Name: ")
    marks = input("Enter Marks: ")
    with open(file_name, "a") as f:
        f.write(roll + "," + name + "," + marks + "\n")
    print("Student added successfully!\n")
def search_student():
    roll_search = input("Enter Roll Number to search: ")
    found = False
    with open(file_name, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if data[0] == roll_search:
                print("Student Found!")
                print("Roll:", data[0])
                print("Name:", data[1])
                print("Marks:", data[2])
                found = True
                break
    if not found:
        print("Student not found.\n")
def update_marks():
    roll_update = input("Enter Roll Number to update: ")
    new_marks = input("Enter New Marks: ")
    updated_data = []
    found = False
    with open(file_name, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if data[0] == roll_update:
```

```
            data[2] = new_marks
            found = True
            print("Marks updated successfully!")
    updated_data.append(",".join(data))
    with open(file_name, "w") as f:
        for record in updated_data:
            f.write(record + "\n")
    if not found:
        print("Student not found.\n")
while True:
    print("\n1. Add Student")
    print("2. Search Student")
    print("3. Update Marks")
    print("4. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        add_student()
    elif choice == "2":
        search_student()
    elif choice == "3":
        update_marks()
    elif choice == "4":
        print("Program exited.")
        break
    else:
        print("Invalid choice!")
```

OUTPUT

```
*IDLE Shell 3.8.10*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.8.10 (tags/v3.8.10:3d8993a, May  3 2021, 11:48:03) [MSC v.1928 64 b
Type "help", "copyright", "credits" or "license()" for more information.
>>>
============= RESTART: D:\MCA PYTHON\PROGRAMS\Lab Programs\Pro 7.py ========

1. Add Student
2. Search Student
3. Update Marks
4. Exit
Enter your choice: 1
Enter Roll Number: 25MCA11
Enter Name: Anirudhan A
Enter Marks: 99
Student added successfully!


1. Add Student
2. Search Student
3. Update Marks
4. Exit
Enter your choice: 2
Enter Roll Number to search: 25MCA11
Student Found!
Roll: 25MCA11
Name: Anirudhan A
Marks: 99

1. Add Student
2. Search Student
3. Update Marks
4. Exit
Enter your choice: |
```

RESULT

The given program was executed successfully.

SOURCE CODE

8. Pandas DataFrame

```
import pandas as pd
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank", "Grace"],
    "Gender": ["Female", "Male", "Male", "Male", "Female", "Male", "Female"],
    "Maths": [85, 78, 92, 70, 88, 76, 95],
    "Science": [90, 74, 89, 65, 91, 72, 98],
    "English": [88, 80, 85, 60, 87, 75, 93]
}
df = pd.DataFrame(data)
print("Student DataFrame:\n")
print(df)
print("\nMean Marks:")
print(df[["Maths", "Science", "English"]].mean())
print("\nMaximum Marks:")
print(df[["Maths", "Science", "English"]].max())
print("\nMinimum Marks:")
print(df[["Maths", "Science", "English"]].min())
print("\nStatistical Summary using describe():")
print(df[["Maths", "Science", "English"]].describe())
print("\nGender Count:")
print(df["Gender"].value_counts())
```

OUTPUT

```
IDLE Shell 3.8.10
File  Edit  Shell  Debug  Options  Window  Help
============== RESTART: D:\MCA PYTHON\PROGRAMS\Lab Programs\Pro
Student DataFrame:

      Name  Gender  Maths  Science  English
0    Alice  Female     85       90       88
1      Bob    Male     78       74       80
2  Charlie    Male     92       89       85
3    David    Male     70       65       60
4      Eva  Female     88       91       87
5    Frank    Male     76       72       75
6    Grace  Female     95       98       93

Mean Marks:
Maths      83.428571
Science    82.714286
English    81.142857
dtype: float64

Maximum Marks:
Maths      95
Science    98
English    93
dtype: int64

Minimum Marks:
Maths      70
Science    65
English    60
dtype: int64

Statistical Summary using describe():
            Maths     Science    English
count    7.000000    7.000000   7.000000
mean    83.428571   82.714286  81.142857
std      9.089502   12.243560  10.976165
min     70.000000   65.000000  60.000000
25%     77.000000   73.000000  77.500000
50%     85.000000   89.000000  85.000000
75%     90.000000   90.500000  87.500000
max     95.000000   98.000000  93.000000

Gender Count:
Gender
Male       4
Female     3
```

RESULT

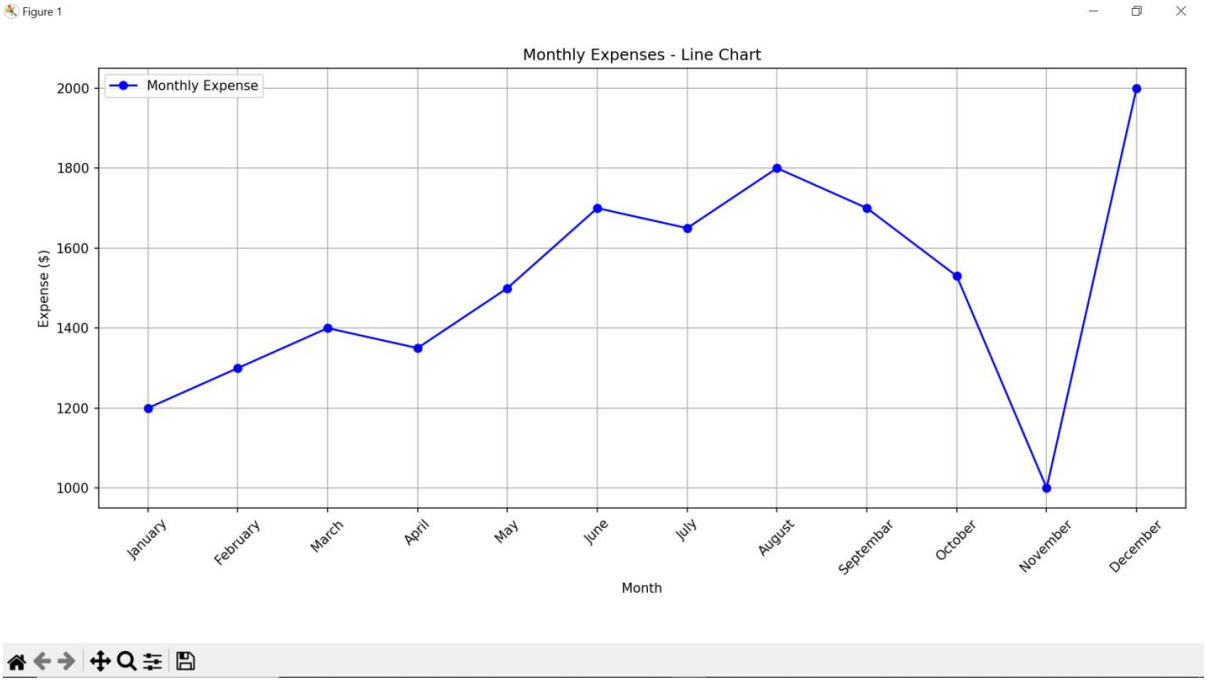
The given program was executed successfully.

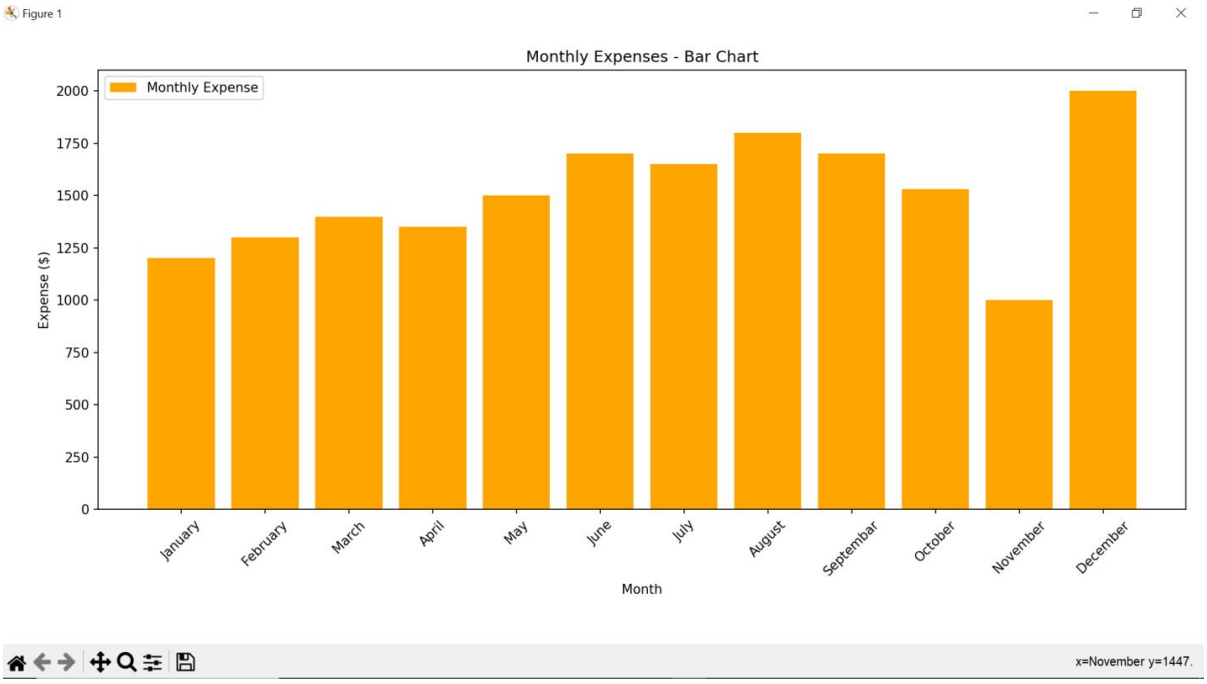
SOURCE CODE

9. Data visualization

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv('expense.csv')
print(data.head())
plt.figure(figsize=(10, 5))
plt.plot(data['Month'], data['Expense'], marker='o', linestyle='-', color='b', label='Monthly Expense')
plt.xlabel('Month')
plt.ylabel('Expense ($)')
plt.title('Monthly Expenses - Line Chart')
plt.xticks(rotation=45)
plt.legend()  plt.grid(True)
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 5))
plt.bar(data['Month'], data['Expense'], color='orange', label='Monthly Expense')
plt.xlabel('Month')
plt.ylabel('Expense ($)')
plt.title('Monthly Expenses - Bar Chart')
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT





RESULT

The given program was executed successfully.

SOURCE CODE

10. File Manipulation

```
input_file = "hello.txt"
output_file = "long_words.txt"
line_count = 0
word_count = 0
long_words = []
with open(input_file, "r") as f:
    for line in f:
        line_count += 1
        words = line.strip().split()
        word_count += len(words)
        for word in words:
            if len(word) > 5:
                long_words.append(word)
with open(output_file, "w") as f:
    for word in long_words:
        f.write(word + "\n")
print("Total lines:", line_count)
print("Total words:", word_count)
print("Long words saved to", output_file)
```

OUTPUT

```
IDLE Shell 3.8.10
File  Edit  Shell  Debug  Options  Window  Help
Python 3.8.10 (tags/v3.8.10:3d8993a, May  3 2021, 11:48:
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for m
>>>
============ RESTART: D:\MCA PYTHON\PROGRAMS\Lab Program
Total lines: 3
Total words: 12
Long words saved to long_words.txt
>>>
```

RESULT

The given program was executed successfully.

SOURCE CODE

11. Graphical Calculator Using Tkinter

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()
root.title("Simple Calculator")
root.geometry("300x400")
expression = ""
entry_text = tk.StringVar()
def press(num):
    global expression
    expression += str(num)
    entry_text.set(expression)
def equal():
    global expression
    try:
        result = str(eval(expression))
        entry_text.set(result)
        expression = result
    except ZeroDivisionError:
        messagebox.showerror("Error", "Cannot divide by zero")
        expression = ""
        entry_text.set("")
    except:
        messagebox.showerror("Error", "Invalid input")
        expression = ""
        entry_text.set("")
def clear():
    global expression
    expression = ""
    entry_text.set("")
entry = tk.Entry(root, textvariable=entry_text, font=("Arial", 18), justify="right")
entry.pack(fill="both", padx=10, pady=10)
```

```
buttons =
    [ ['7','8','9','/'],
    ['4','5','6','*'],
    ['1','2','3','-'],
    ['0','C','=','+']
]
for row in buttons:
    frame = tk.Frame(root)
    frame.pack(expand=True, fill="both")
    for btn in row:
        if btn == "=":
            action = equal
        elif btn == "C":
            action = clear
        else:
            action = lambda x=btn: press(x)
        tk.Button(
            frame,
            text=btn,
            font=("Arial", 14),
            command=action
        ).pack(side="left", expand=True, fill="both")
root.mainloop()
```

OUTPUT

RESULT

The given program was executed successfully.

SOURCE CODE

12. Django Web Application

```
from django.shortcuts import render
def login_view(request):
    message = ""
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
    if username == "admin" and password == "1234":
        message = "Login Successful"
    else:
        message = "Invalid Username or Password"
return render(request, "login.html", {"message": message})
#myapp/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.login_view, name="login"),
]
#django_web/urls.py
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp.urls')),
]
#templates/login.html
<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
<h2>Login Form</h2>
<form method="post">
    {% csrf_token %}
    Username:
    <input type="text" name="username"><br><br>

    Password:
    <input type="password" name="password"><br><br>
    <input type="submit" value="Login">
</form>
<p>{{ message }}</p>
</body>
</html>
```

OUTPUT

User Login

Username

Password

Login

Login Successful

RESULT

The given program was executed successfully.